

G05 / ASSESSMENT

G05 GATE A

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Case A - cold-chain event pipeline 3

Practical evidence A 3

G05 GATE A - REQUIRED IMPLEMENTATION 3

Required runnable integration evidence A 3

Critical failure rules A 4

Read this first

Formal stage-boundary assessment after M14. Closed-book. Untimed mastery first. Save the full attempt before opening Review A.

Case A - cold-chain event pipeline

1. Architecture: Raw IoT events arrive continuously. Explain the roles of Kafka, Spark Structured Streaming, Parquet/Delta and Bronze/Silver/Gold without collapsing them into one product.
2. Kafka design: Choose the message key for sensor events when per-device order matters. State partition-ordering and hotspot implications.
3. Consumer correctness: A Python alert consumer calls an external SMS API. Design progress/idempotency so restart does not send duplicate alerts.
4. Replay: A transformation bug affected 20 minutes of Silver. Design a replay using Kafka offsets/checkpoint or durable Bronze evidence without corrupting Gold.
5. Delta: Explain how MERGE/idempotent keys and table history help repair the affected window.
6. Spark: State what a Structured Streaming checkpoint owns and why deleting it is not a first troubleshooting step.
7. Operations: Partition 4 lag grows while others are healthy. Give evidence-driven diagnosis and one recovery validation.

Practical evidence A

1. Use the supplied G05_A_events.jsonl. Identify duplicate event_id, late event and malformed record. Write the expected valid/quarantine/replay treatment.
2. Write three reconciliation invariants for the pipeline (for example input = valid + quarantine, unique event IDs, and one business aggregate).
3. Write a one-page incident note: symptom, hypothesis, evidence, safe action, validation, rollback boundary.

G05 GATE A - REQUIRED IMPLEMENTATION

Required runnable integration evidence A

Written reasoning alone is not enough for this stage gate. After the conceptual answers, implement a small program in Python/Spark/your approved local stack that reads the supplied G05_A_events.jsonl and produces durable valid + quarantine evidence.

Required behavior: malformed not-json is quarantined with a reason; duplicate E2 produces one business effect; late E3 follows an explicit late-event policy rather than disappearing; raw source stays unchanged.

Write G05_A_RUNTIME_EVIDENCE.json (or equivalent) containing observed_input=5, malformed/quarantine count, unique accepted event IDs, duplicate treatment, late-event treatment, and one business aggregate. The invariant observed input = handled valid records including duplicates/replays + quarantine must be explained with your chosen

dedupe boundary.

Deliberate failure: make the valid sink unwritable or inject a controlled write failure. Prove the valid event is not reclassified as bad source data and that its progress is not falsely committed. Restore the sink and rerun.

Changed retry: modify one valid temperature and add a new event ID. Rerun with review closed and prove exactly the intended business state changes.

Critical failure rules A

Fail the gate if the implementation loses raw evidence, commits a failed valid write, silently drops malformed/late input, duplicates a business effect on replay, or cannot reconcile its bounded input.

Runtime proof and conceptual reasoning are scored separately. A correct essay cannot compensate for destructive implementation behavior.